TCP Client-Server Application

Design, Reliability Improvements & Testing

Project: Computer Networks course project (2023), reliability revision (2026)

Language: C++17 | **Environment:** Linux / Ubuntu on WSL 2

Repository: https://github.com/HLin34/Socket_programming

Author: Hung-Chang (Henry) Lin

1. Executive Summary

This project implements a menu-driven TCP client-server application using POSIX sockets. The original 2023 course implementation supported IPv4 DNS resolution, a student-ID-to-email lookup backed by a local text file, and an interactive quit option. In the 2026 Phase 1 revision, the core functionality was retained while the network protocol, input safety, socket cleanup, and local test data were improved.

2. System Architecture

The C++ client connects to a server bound to **127.0.0.1:1234**. The server processes one connection at a time and responds to three menu choices: **1 - DNS lookup**, **2 - student lookup**, and **3 - quit**. Both programs use the shared `protocol.hpp` message helpers. The server loads fictitious student records from `students_sample.txt` into an `std::unordered_map`.

Message flow

```
Client                  Server (127.0.0.1:1234)
  | ---- TCP connect ----> |
  | <---- menu prompt ---- |
  | ---- choice 1/2 -----> |
  | <---- input prompt --- |
  | ---- input value ----> |
  | <---- response ------- |
  | ---- choice 3 -------> |
  |     disconnect         |
```

3. Network Protocol

The revised implementation frames each application message using a four-byte unsigned length field in network byte order followed by the message payload.

```
[ uint32 payload length (big-endian) ][ payload bytes ]
```

`send_all()` and `receive_all()` loop until the requested number of bytes is transferred or a connection error occurs. Incoming messages are limited to **4,096 bytes**. This makes the application independent of how TCP splits or coalesces the underlying byte stream. The revised protocol is incompatible with the legacy unframed client/server: both sides must use the new implementation.

4. Implementation Changes

Concern	Original implementation	Phase 1 implementation
Receiving data	Fixed 256-byte buffer and unchecked `read()`	Length-prefixed messages and complete reads
Sending data	Assumed a single `write()` was sufficient	`send_all()` handles partial sends

Concern	Original implementation	Phase 1 implementation
User input	Unbounded `scanf("%s", ...)`	`std::getline()`
DNS	`gethostbyname()`	`getaddrinfo()` and `inet_ntop()` for IPv4
Student lookup	`std::vector<Student>` linear scan	`std::unordered_map<std::string, std::string>`
Client cleanup	Accepted socket closed outside the server loop	Accepted socket closed after each session
Network exposure	`INADDR_ANY`	Loopback-only binding
Test data	Course-specific records	Fictitious sample records

5. Validation

Local compilation: The revised `server.cpp` and `client.cpp` compiled under Ubuntu / WSL using C++17 with `-Wall -Wextra -Wpedantic` and no warnings in the observed build.

Interactive tests observed:

Test	Input / scenario	Result
DNS lookup	`localhost`	Returned `127.0.0.1`
Existing student	`DEMO001`	Returned `alex@example.com`
Missing student	`Unknown`	Displayed `No such student ID`
Invalid menu choice	`9`	Returned an error and continued
Quit and reconnect	`3`, then launch a new client	Second client connected successfully

An earlier controlled loopback test also exercised an oversized incoming frame and subsequent reconnection. These tests demonstrate basic functionality, **not** complete security, robustness, or concurrency coverage.

6. How to Build and Run

```
g++ -std=c++17 -Wall -Wextra -Wpedantic server.cpp -o server
g++ -std=c++17 -Wall -Wextra -Wpedantic client.cpp -o client
```

Open two Ubuntu terminals in the repository root:

```
# Terminal 1
./server

# Terminal 2
./client
```

By default, the server loads `students_sample.txt`. The project is designed for local demonstration only; do not expose the test service publicly.

7. Limitations and Future Work

• The server is **single-threaded** and handles clients sequentially; concurrent connections and load testing are not yet implemented.

- Socket ownership uses explicit `close()` calls rather than a dedicated RAII wrapper.
- There is no authentication, encryption, persistent database, or production logging.
- Formal unit/integration test automation, sanitizer testing, and CI remain future improvements.
- IPv4 is used for the current DNS response and TCP listener.

8. Data Privacy and Provenance

The original 2023 course report and legacy `query.txt` may contain personal names, student IDs, or email addresses and should not be included in a public-facing portfolio without review. The current demonstration uses synthetic records. Removing a file from the latest Git commit does **not** erase it from previous public Git history; historical exposure should be reviewed separately.

Project provenance: The initial version was developed as a university computer networks final project in 2023. Phase 1 represents a later reliability and documentation revision, not an original 2023 feature set.